

Mise en place d'un serveur PXE

Installer automatiquement un OS Linux sur une machine vierge à travers une connexion réseau.

Table des matières

Mise en place d'un serveur PXE.....	1
Présentation.....	1
Préparation de la VM serveur.....	2
Clonage de la machine serveur.....	4
Insertion des scripts dans la machine serveur.....	5
Exécution des scripts.....	7
Configuration en bridge, mise à jour et ajouts logiciels.....	7
Configuration en réseau host-only.....	8
Activation DHCP.....	9
Création machine cliente vide.....	10
Démarrage machine cliente.....	10
Redémarrage machine cliente.....	12
Evolutions.....	12
Fonctions shell.....	13
Saisielp.....	13
ConfigMask.....	13
ajoutlogi.....	14
ConfigGTW.....	15
Scripts utilisés.....	16
Script principal.....	16
Script fonctions.....	26
Fichier de configuration mini.cfg.....	36
Fichier de configuration maxi.cfg.....	37

Présentation

La réalisation de cette procédure est orientée vers la portabilité, en la testant sur d'autres versions de linux.

Elle est exposée ici à partir d'une distribution Alma Linux 9 minimale, depuis un fichier *mini.cfg*.

Elle peut être appliquée pour une distribution Alma Linux 9 complète, depuis un fichier *maxi.cfg*.

Des essais ont été menés sur Fedora, mais plusieurs différences de réglages n'ont pas permis d'aboutir.

Les captures d'écrans sont parfois réduites, il peut être nécessaire de zoomer pour les lire en détail.

Préparation de la VM serveur

AlmaLinux 9.5

Install AlmaLinux 9.5
Test this media & install AlmaLinux 9.5
Troubleshooting

Press Tab for full configuration options on menu items.

Click in the virtual screen to send keystrokes

Install CentOS 8 64-bit as you would on a physical computer. When you are done and the operating system boots up, click "I Finished Installing".

I Finished Installing Help

To direct input to this VM, click inside or press Ctrl+G.

Virtual Machine Settings

Hardware Options

Device	Summary
Memory	2 GB
Processors	2
Hard Disk (NVMe)	20 GB
CD/DVD (IDE)	Auto detect
Network Adapter	Bridged (Automatic)
USB Controller	Present
Sound Card	Auto detect
Printer	Present
Display	Auto detect

Device status
☐ Connected
☒ Connect at power on

Connection
☐ Use physical drive:
Auto detect
☒ Use ISO image file:
E:\iso\AlmaLinux-9.5-x86_64-minimal.iso Browse... Advanced...

AlmaLinux 9.5

BIENVENUE DANS ALMALINUX 9.5.

Quelle langue souhaitez-vous utiliser durant le processus d'installation?

Français	French	Français (France)
العربية	Arabic	Français (Canada)
English	English	Français (Belgique)
Deutsch	German	Français (Suisse)
日本語	Japanese	Français (Luxembourg)
中文	Mandarin Chinese	
Русский	Russian	
Español	Spanish	
Afrikaans	Afrikaans	
አማርኛ	Amharic	
অসমীয়া	Assamese	
Астуріян	Asturian	
Беларуская	Belarusian	
Български	Bulgarian	
বাংলা	Bangla	
Català	Catalan	

Quitter Configurer

AlmaLinux

RÉSUMÉ DE L'INSTALLATION

INSTALLATION DE ALMALINUX 9.5

LOCALISATION

Claavier
Français (Français (variante))
Langues prises en charge
Français (France)
Heure et date
Format horaire Europe/Paris

LOGICIEL

Source d'installation
Media local
Sélection de logiciels
AlmaLinux

SYSTÈME

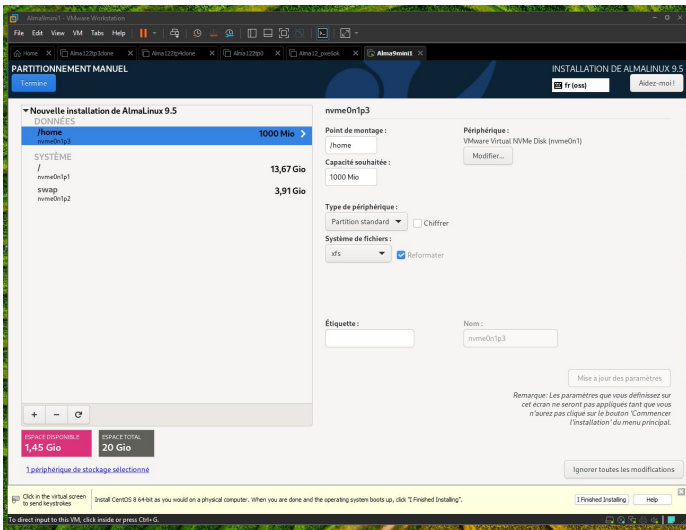
Destination de l'installation
Partitionnement automatique
all-disk
KDUMP
Edump est activé
Réseau et nom d'hôte
Connexion en033
Profil de sécurité
Aucun profil sélectionné

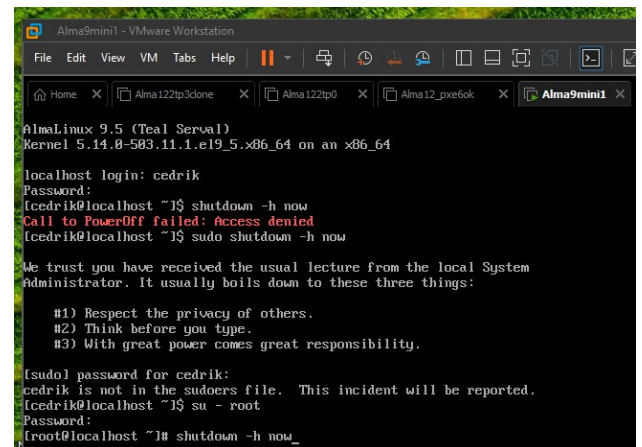
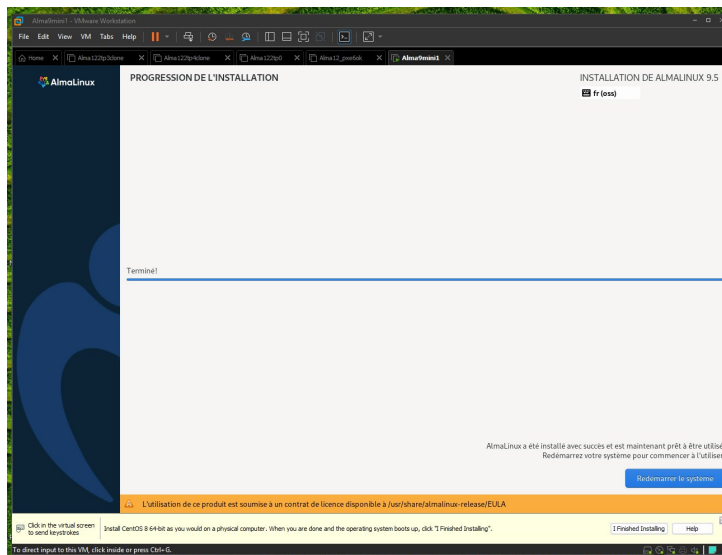
UTILISATEURS

Mot de passe administrateur
Le compte root est désactivé
Création de l'utilisateur
Aucun utilisateur ne sera créé

Quitter Commencer l'installation

Vous devez sélectionner au moins un élément avant de passer à l'étape suivante.

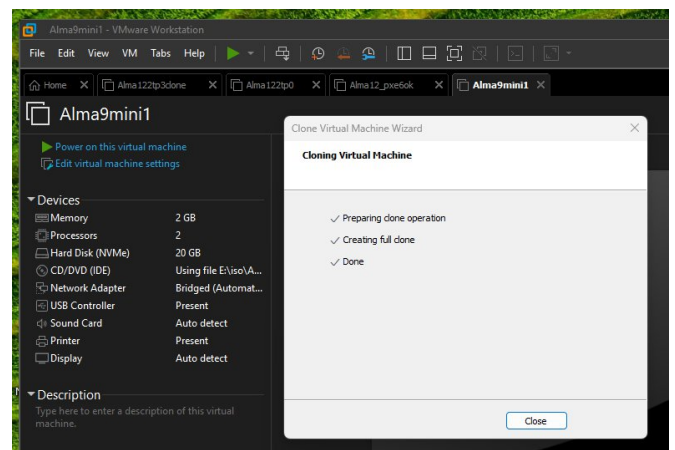
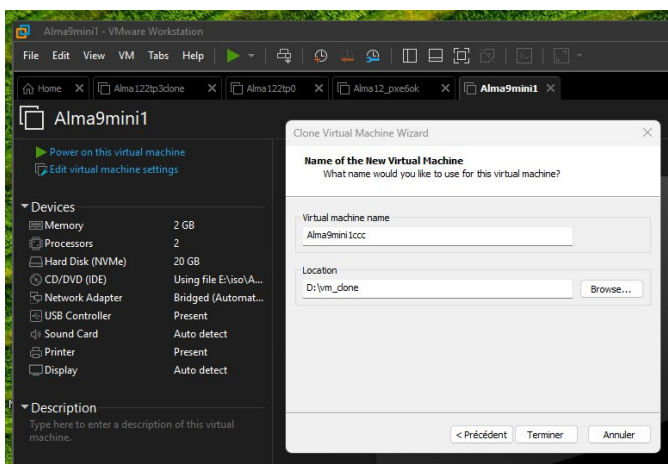




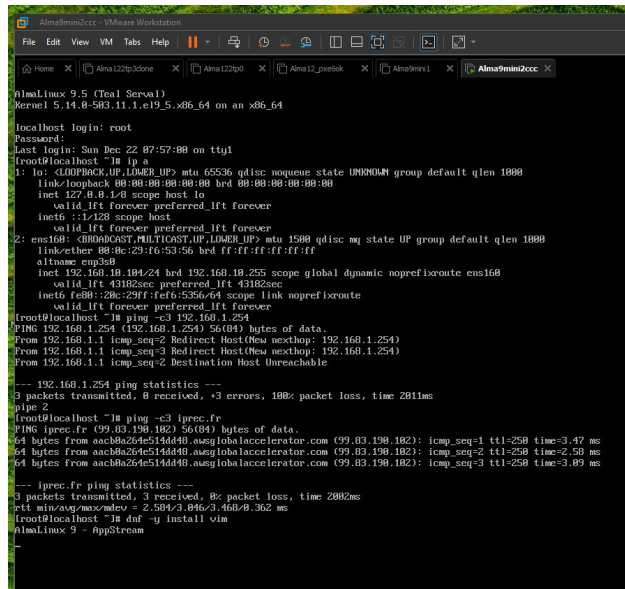
Après redémarrage de cette VM, la version minimale s'ouvre sur le terminal par défaut.

Par essence, les commandes ou fonctions disponibles sont réduites. Certaines pourront manquer

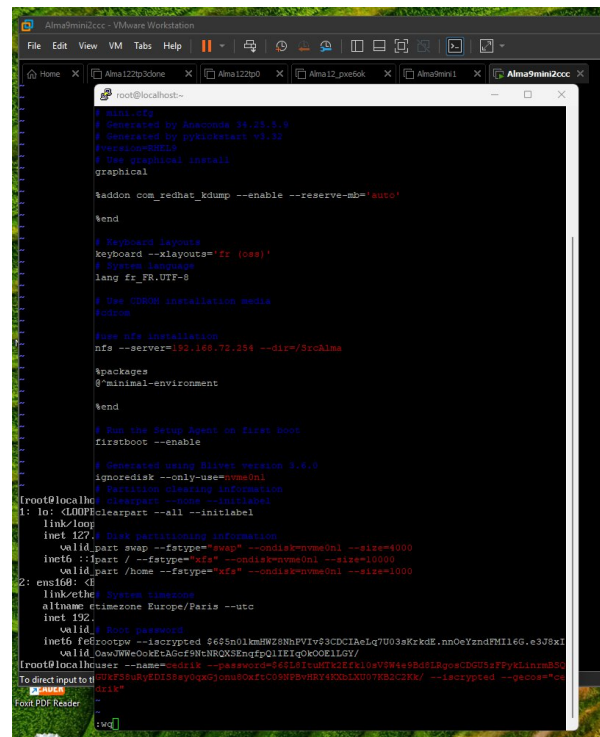
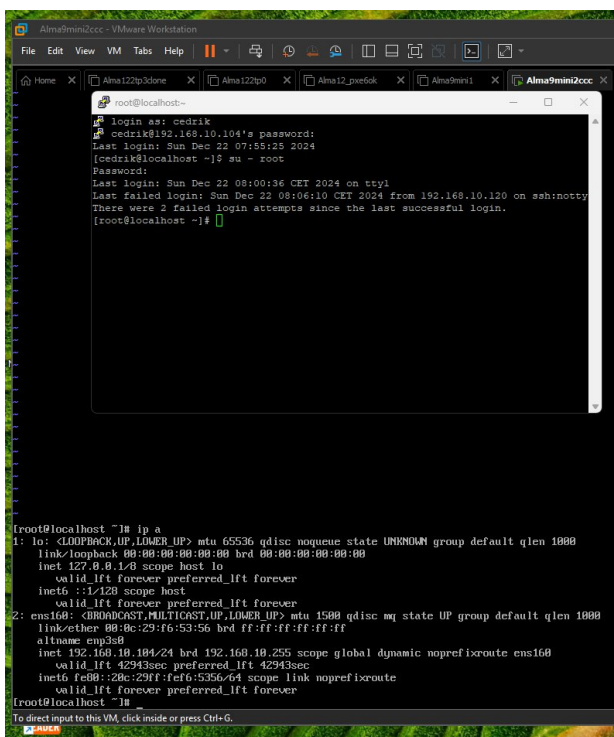
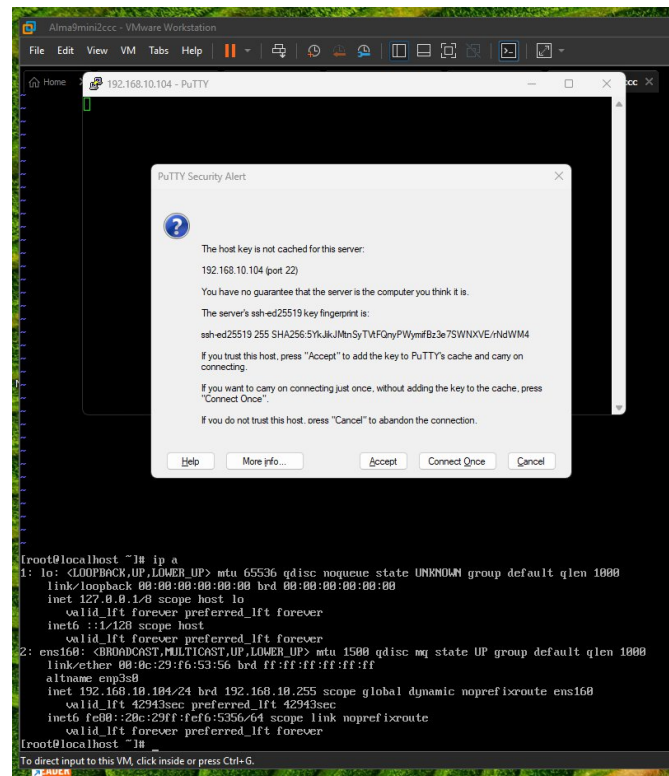
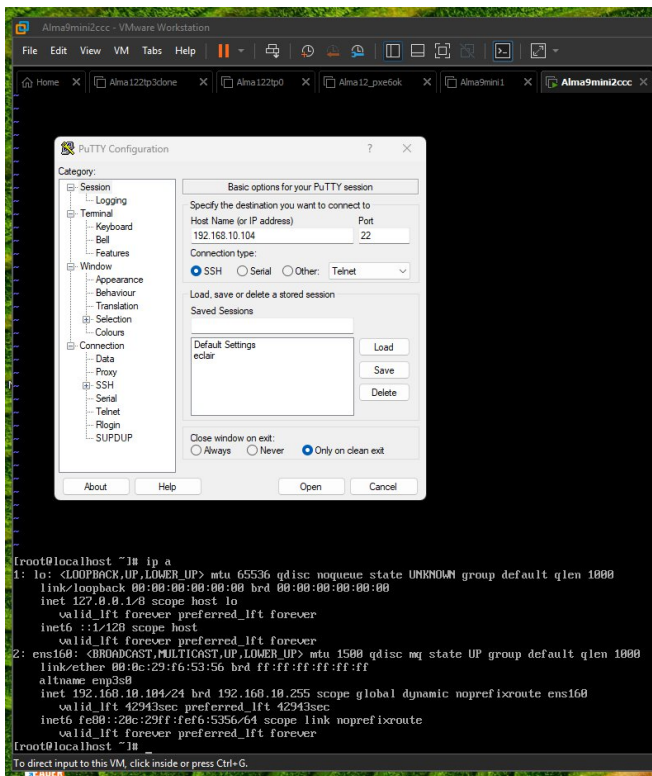
Clonage de la machine serveur

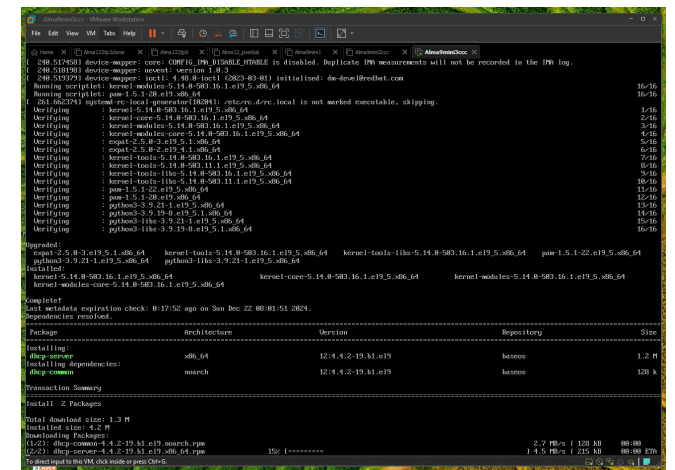
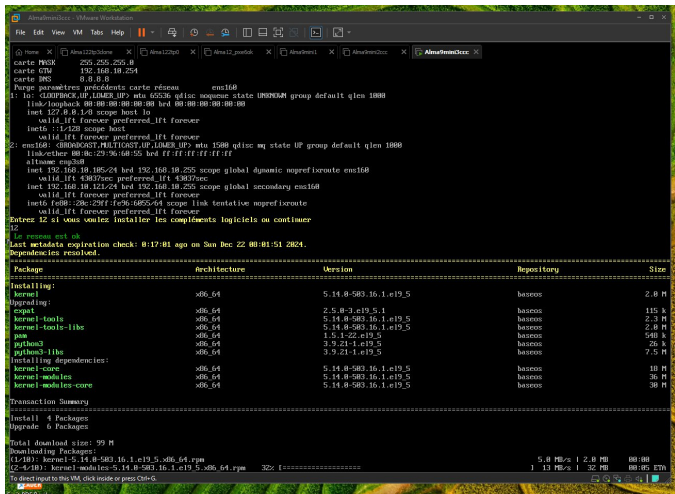
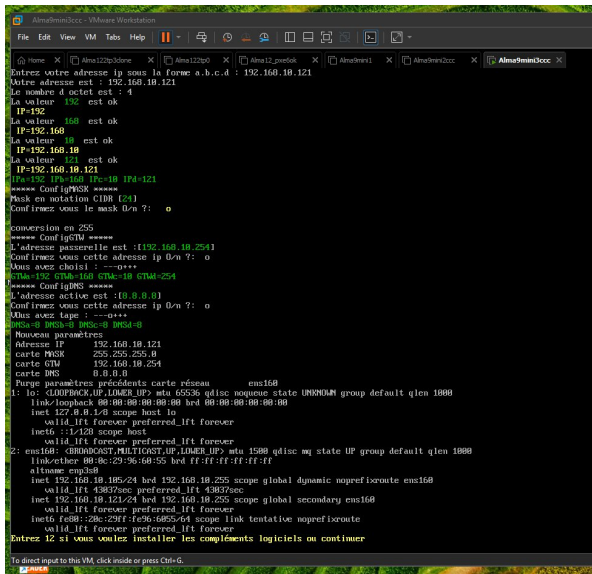
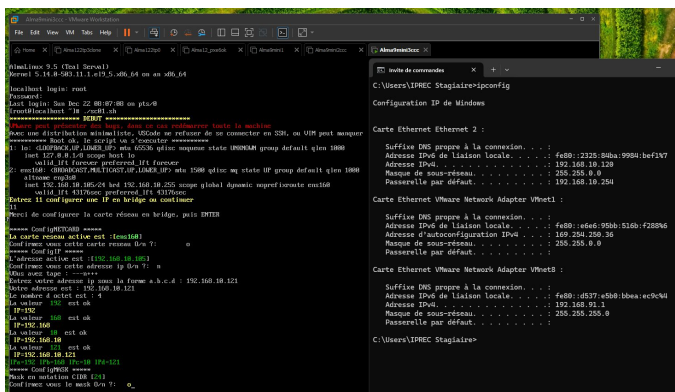


Par sécurité, et pour éviter de tout refaire en cas de mauvais réglages, la vm est clonée.

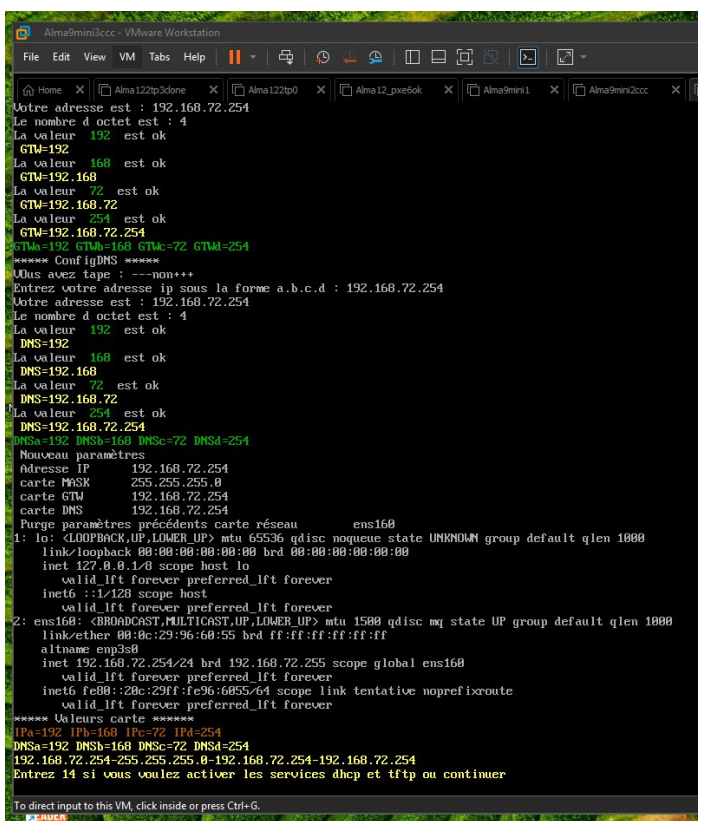
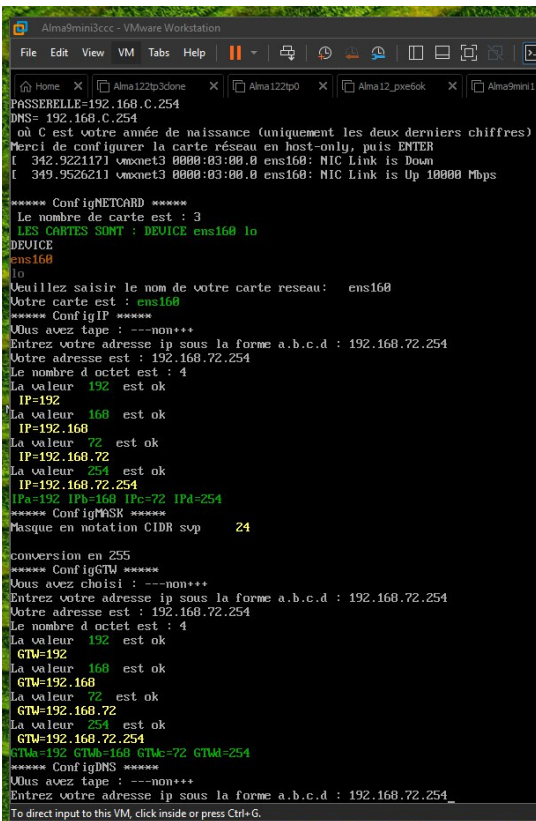
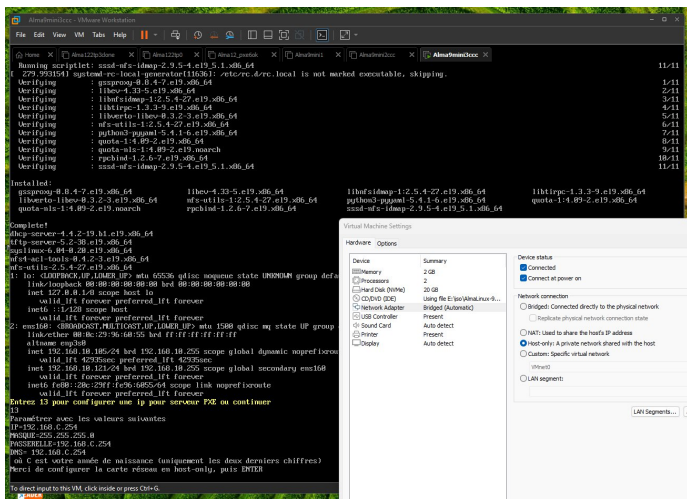


Insertion des scripts dans la machine serveur

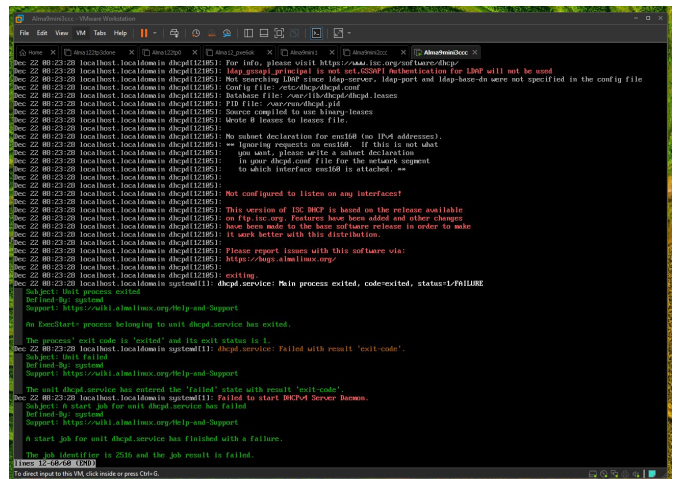
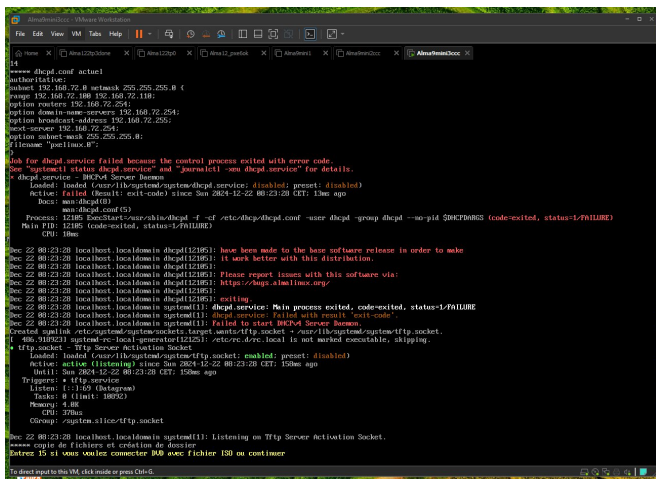




Configuration en réseau host-only



Activation DHCP

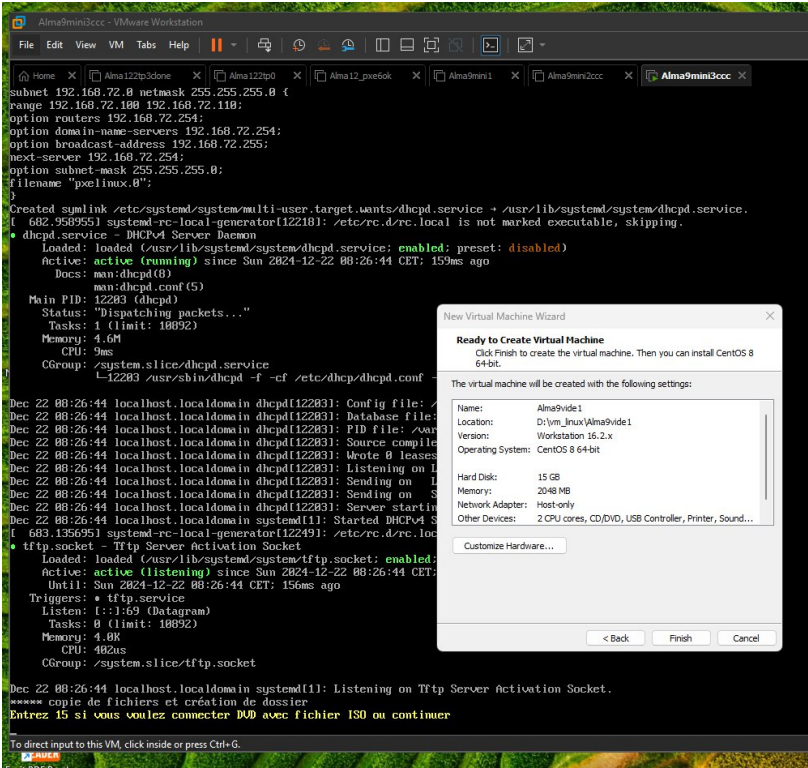


Le service n'a pas démarré au premier déclenchement, la commande `journalctl -xeu dhcpd.service` nous indique que la carte réseau n'est pas prise correctement en compte.

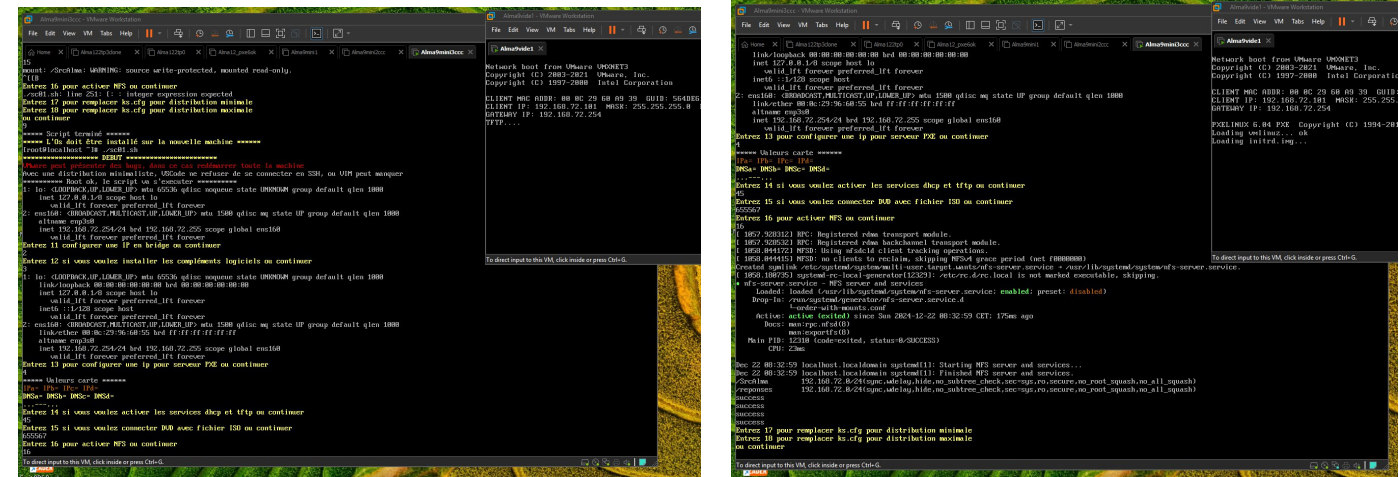
Le script de configuration est interrompu et l'étape de configuration pour PXE refaite attentivement.

Tout rentre dans l'ordre pour passer aux étapes suivantes qui défilent très vite.

Création machine cliente vide

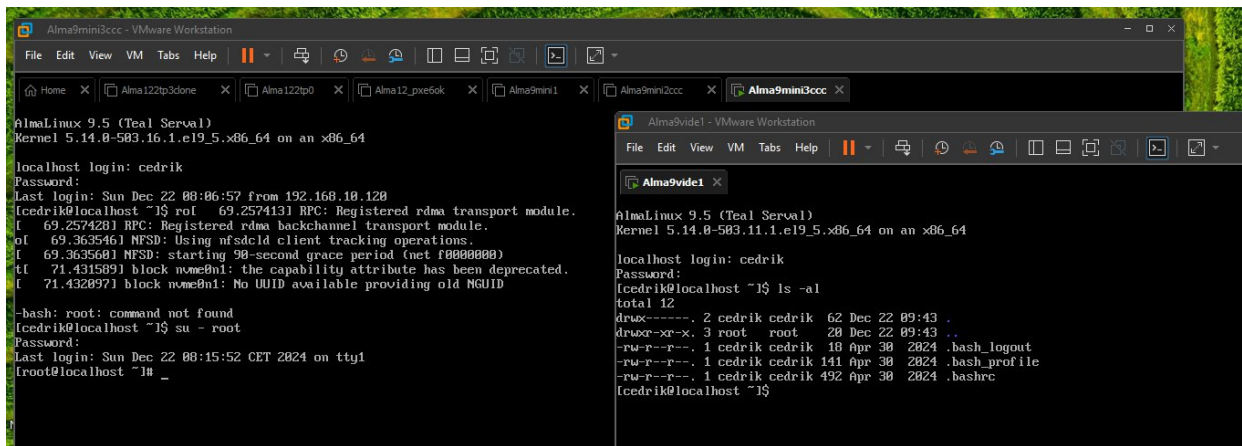


Démarrage machine cliente



L'installation se déroule correctement en automatique

Redémarrage machine cliente



La machine cliente démarre correctement sur Alma linux minimal, donc ici en mode console, sans GUI.

Evolutions

1 Jusqu'à présent, les scripts ne fonctionnent totalement que sur la machine de l'iprec.

Je n'ai pas encore identifié la modification de configuration, qui permettrait que le script fonctionne complètement sur une machine à domicile avec VMware.

C'est la prise en compte du ks.cfg qui échoue, donc l'installation finale de la machine cliente n'aboutit pas dans ce cas.

2 L'automatisation de l'adaptation du fichier ks.cfg est une étape importante.

Elle peut se faire notamment avec l'utilisation de la commande sed, par exemple pour remplacer le cdrom par nfs comme source des fichiers d'installation :

Vérifie si la ligne contient "cdrom" et remplace par la nouvelle ligne

```
ks_file="ks.cfg"
if grep -q "cdrom" "$ks_file"; then
    sed -i 's|cdrom|nfs --server=192.168.72.254 --dir=/SrcAlma|' "$ks_file"
    echo "La ligne 'cdrom' a été remplacée par 'nfs --server=192.168.72.254 --dir=/SrcAlma'."
else
    echo "La ligne 'cdrom' n'a pas été trouvée dans le fichier."
fi
```

Ainsi, il n'y a plus besoin de fichiers spécifiques *mini.cfg* et *maxi.cfg*.

3 L'application de ce script à d'autres distributions Linux est l'étape la plus utile.

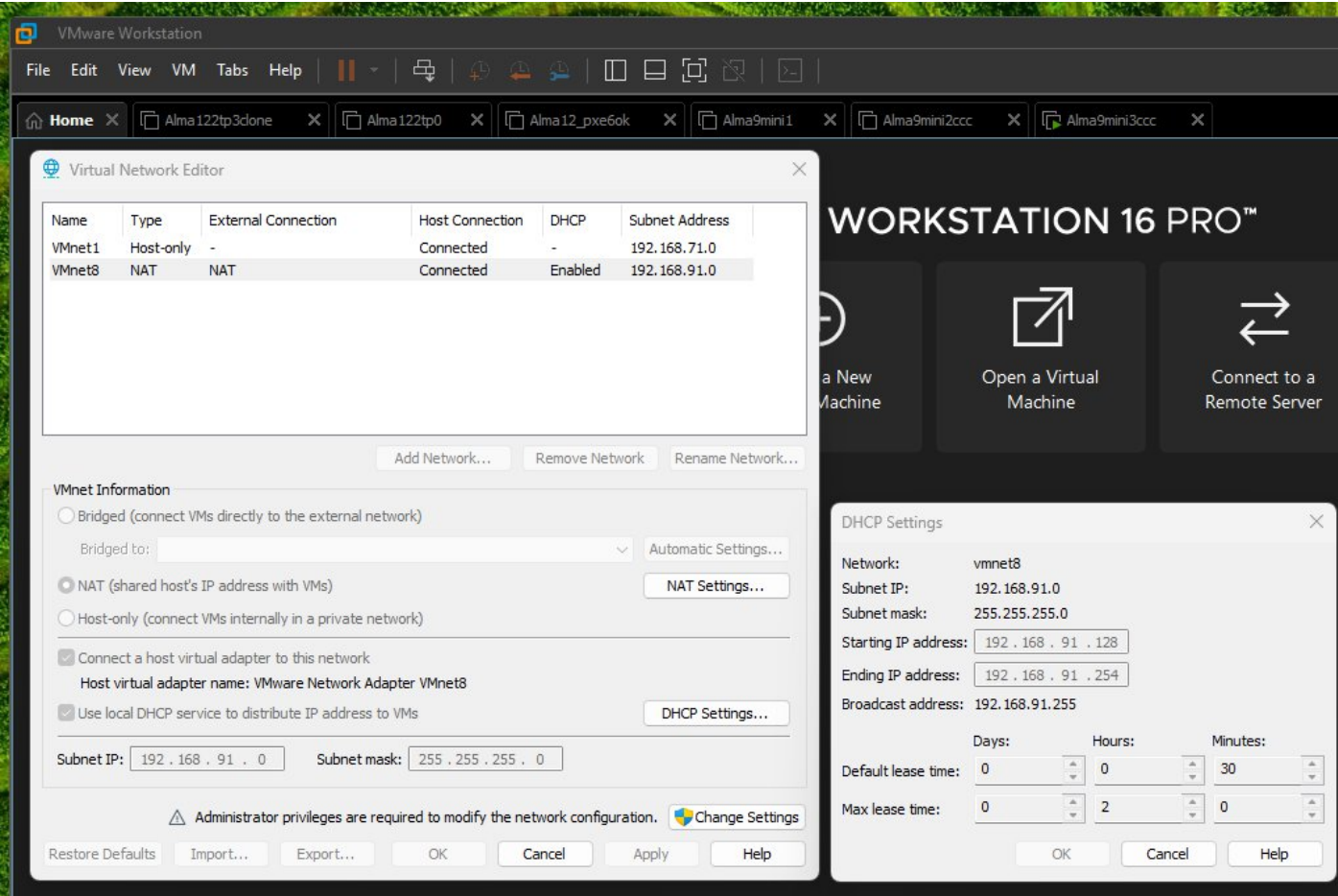
Pour cela, il faut prendre le temps de tester et comprendre les spécificités de chaque distribution Linux, puis ajuster les paramètres des fichiers ks.cfg. Idéalement encore en automatisant cette adaptation.

4 Compléments pour réseau à domicile :

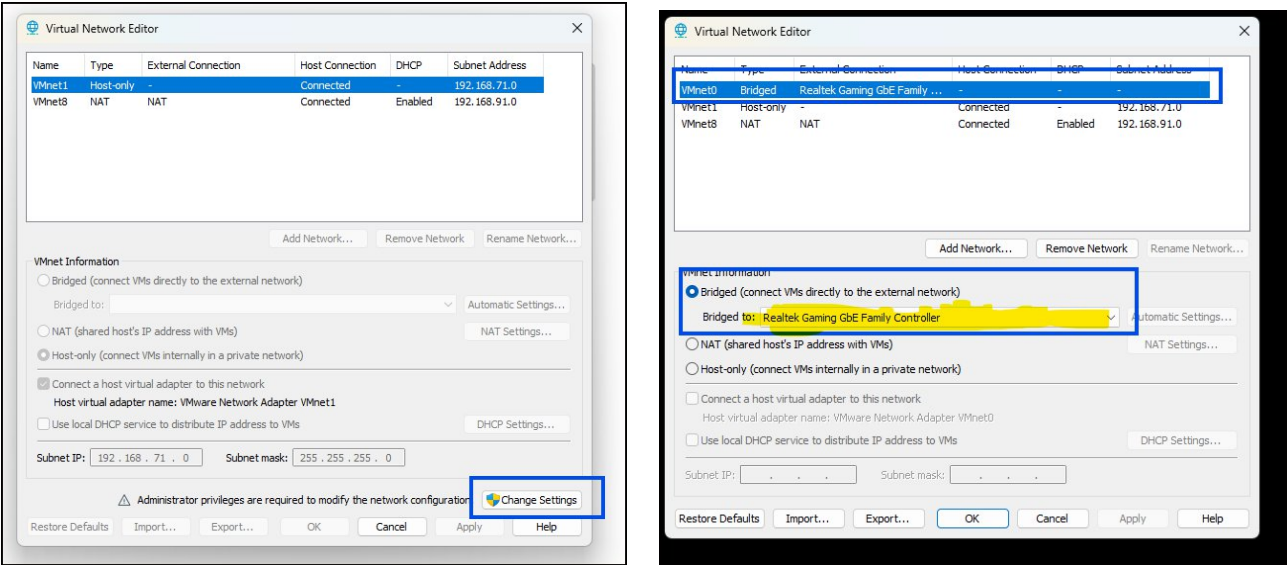
Modifier le DHCP de VmWare : Exemple avec IT connect

<https://www.youtube.com/watch?v=hj-deoZA4do>

Illustrations des configs lprec :



Exemple d'Ali :



Fonctions shell

SaisieIp

```
34 SaisieIp()
35 {
36     echo -e "Entrez votre adresse ip sous la forme a.b.c.d :\t\c"
37     read ip
38
39     echo "Votre adresse est : $ip"
40     IFS="."
41     while true
42     do
43         TabIp=($ip)
44         NbOctets=${#TabIp[*]}
45
46         echo "Le nombre d octet est : $NbOctets"
47         if [ $NbOctets -ne 4 ]
48         then
49             echo -e "${rouge}$ip ${blanc} Veuillez saisir une adresse ip valide (4 octets) :\t\c"
50             read ip
51         else
52             octeta=${TabIp[0]}; octetb=${TabIp[1]}; octetc=${TabIp[2]}; octetd=${TabIp[3]}
53             IFS=" "
54             break
55         fi
56         echo "Votre adresse est : $ip"
57     done
58 }
59
```

L'utilisateur saisit une adresse ip, qui est réaffichée.

On définit le. Comme caractère de séparation des valeurs dans une chaîne d'information.

Une boucle tourne tant qu'il n'y a pas de test négatif à l'intérieur.

Les 4 valeurs de l'adresse ip sont isolées dans *TabIp* et le nombre d'octets est écrit dans *NbOctets*

NbOctets est affiché à l'écran et une vérification qu'il n'est pas différent de 4, ce qui basculerait dans le cas du *then* : Une nouvelle adresse ip est demandée

Sinon, les valeurs de chaque octet sont transférées dans 4 variables standard *octeta...* *Octetd*

Le caractère de séparation est remis à « espace ».

On quitte la boucle, puis on affiche à l'écran la valeur de l'octet, avant de fermer la fonction.

ConfigMask

```
162 ConfigMASK()
163 {
164     cidr=$(ip -4 addr | grep $NetCard | grep inet | awk -F '[ /]' '{print $7}')
165     if [ -n "$cidr" ]
166     then
167         echo -e "Mask en notation CIDR [${vert}$cidr${blanc}]"
168         echo -e "Confirmez vous le mask O/n ?:${jaune}\t\c"
169         read reponse
170         echo -e "${blanc}"
171     else
172         reponse="non"
173     fi
174     case "$reponse" in
175         "" | [oO] ) SaisieMASK $cidr ;;
176         * ) echo -e "Masque en notation CIDR svp ${jaune}\t\c"
177             read cidr
178             echo -e "${blanc}"
179             SaisieMASK $cidr ;;
180     esac
181 }
```

La variable *cidr* récupère depuis la commande *ip addr* la valeur en 7^{ème} position.

Elle extrait aussi les valeurs écrites dans *NetCard* et *inet*.

Le test *if* vérifie que *cidr* est non vide, dans ce cas, elle demande confirmation à l'utilisateur s'il veut valider cette valeur pour le masque.

Sinon, la variable *reponse* devient non. Ensuite, selon la valeur de *reponse*:

Les caractères O ou o transfèrent la valeur de *cidr* pour le traitement par *SaisieMASK*, sinon, une nouvelle valeur de *cidr* est demandée avant d'être envoyée à *SaisieMASK*.

ajoutlogi

```
77 ajoutlogi()
78 {
79     echo -e "${vert} Vérification de la connexion à internet${jaune}"
80     sites="iprec.fr yahoo.fr"
81     erreurs=0
82     for site in $sites
83     do
84         ping -c4 $site >/dev/null 2>&1
85         ((erreurs+=${?}))
86     done
87
88     if [ $erreurs -eq 0 ]
89     then
90         echo -e "${vert} Le reseau est ok${jaune}"
91     else
92         echo -e "${rouge} Probleme de reseau ${blanc}"
93         exit13
94     fi
95     echo -e "${vert} Mise à jour de l'OS${jaune}"
96     yum -y update
97     echo -e "${vert} Ajout des packets nécessaires${blanc}"
98     packages="dhcp-server.x86_64 tftp-server.x86_64 syslinux.x86_64 nfs4-acl-tools.x86_64 nfs-utils"
99     erreurs=0
100    for packet in $packages
101    do
102        yum -y install $packet
103        ((erreurs+=${?}))
104    done
105
106    # ameliorer si erreur !=0
107    echo -e "${vert} Vérification des packets${blanc}"
108    erreurs=0
109    for packet in $packages
110    do
111        rpm -q $packet
112        ((erreurs+=${?}))
113    done
114    echo -e "${vert} Vérification SELinux${blanc}"
115    sestatus
116 }
```

Cette fonction permet de regrouper toutes les opérations d'ajout de couches logicielles nécessaires pour les scripts

Elle commence par vérifier que la connexion réseau permet d'atteindre le site *iprec.fr* par internet.

La première boucle *do/done* accumule les valeurs d'erreur dans la variable *erreurs*.

Le *if* teste si le résultat reste nul, donc le *ping* fiable, et le réseau exploitable.

La mise à jour de l'OS est déclenchée avec *yum -y install*

Puis les paquets requis pour le script sont dans la variable *packages*.

Nouvelle vérification qu'il n'y pas d'erreur de traitement pendant l'appel des paquets successifs.

Enfin, la commande *rpm* dans la dernière boucle *do/done* permet d'afficher si les paquets sont bien

opérationnels.

La dernière commande `sestatus` permet de vérifier l'état et donc si *SELinux* est bien installé et opérationnel.

ConfigGTW

```
183 ConfigGTW()
184 {
185     #Recherche de l'adresse GTW active
186     ip=$(ip -4 route show | grep "^default" | awk 'NR == 1 {print $3}')
187     # ip=$(ip -4 addr | grep $NetCard | grep inet | awk -F "[ /]" '{print $6}')
188     if [ -n "$ip" ]
189     then
190         echo -e "L'adresse passerelle est :[${vert}]$ip[${blanc}]"
191         echo -e "Confirmez vous cette adresse ip O/n ? : \t\c"
192         read reponse
193     else
194         reponse="non"
195     fi
196     echo "Vous avez choisi : ---${reponse}+++"
197     case "$reponse" in
198         "" | [oO] ) IFS="."
199                     TabIp=($ip)
200                     GTWa=${TabIp[0]}; GTWb=${TabIp[1]}; GTWc=${TabIp[2]}; GTWd=${TabIp[3]}
201                     GTW=$GTWa.$GTWb.$GTWc.$GTWd
202                     echo -e "${vert}GTW=$GTW GTWb=$GTWb GTWc=$GTWc GTWd=$GTWd[${blanc}]"
203                     IFS=" ";
204                     * ) SaisieIp
205
206                     # 1 <= octeta <= 254
207                     VerifOctet 1 254 $octeta
208                     octeta=$?
209                     GTW=$octeta
210                     echo -e "${jaune} GTW=$GTW ${blanc}"
211
212                     # 0 <= octetb <= 255
213                     VerifOctet 0 255 $octetb
214                     octetb=$?
215                     GTW+=.$octetb
216                     echo -e "${jaune} GTW=$GTW ${blanc}"
217
218                     # 0 <= octetc <= 255
219                     VerifOctet 0 255 $octetc
220                     octetc=$?
221                     GTW+=.$octetc
222                     echo -e "${jaune} GTW=$GTW ${blanc}"
223
224                     # 1 <= octetd <= 254
225                     VerifOctet 1 254 $octetd
226                     octetd=$?
227                     GTW+=.$octetd
228                     echo -e "${jaune} GTW=$GTW ${blanc}"
229                     GTWa=$octeta; GTWb=$octetb; GTWc=$octetc; GTWd=$octetd
230                     echo -e "${vert}GTW=$GTW GTWb=$GTWb GTWc=$GTWc GTWd=$GTWd[${blanc}]"
231                     IFS=" "
232                 esac
233 }
```

Cette fonction permet de saisir une nouvelle adresse ip pour configurer la passerelle par défaut de la machine. Dans les grandes lignes :

L'adresse ip existante est extraite de la commande route show.
Si elle est non nulle, on demande de la valider ou de la modifier.
En ligne 197, selon la réponse :
Depuis la ligne 198, soit la valeur actuelle de l'ip est décomposée pour être mémorisée afin d'être écrite dans la configuration.
Sinon, depuis la ligne 204, on saisi des nouvelles valeurs pour l'adresse ip :
En ligne 207, on vérifie que l'octet est dans le bon intervalle de valeurs.
En ligne 208, que la saisie ne retourne pas d'erreur.
En ligne 2009, on ajoute cet octet à la variable GTW pour composer l'adresse finale, puis il est affiché.
Le cycle est répété pour les 3 autres octets.
En ligne 230, un affichage récapitulatif termine la fonction, avant de redéfinir l'eospace comme séparateur de chaîne.

Scripts utilisés

Script principal

```
#!/bin/bash

#Variables pour la coloration syntaxique

# set -x

blanc="\033[0m"
noir="\033[30m"
rouge="\033[31m"
vert="\033[32m"
orange="\033[33m"
jaune="\033[1;33m"

ConfigNETCARD()
{
    #Recherche de l'adresse ip active

    NetCard=$(ip -4 route show | grep "^default" | awk 'NR == 1 {print $5}')

    if [ -n "$NetCard" ]
    then
        echo -e "${jaune}La carte reseau active est :[${vert}$NetCard${blanc}]"

        echo -e "Confirmez vous cette carte reseau O/n ?: \t\c"

        read reponse
    else
```

```

reponse="non"

fi

case "$reponse" in
"" | [oO] ) return;;
* ) IFS=" "
    while true
    do
        TabNetCard=$(nmcli -f DEVICE device | tr -d "\n")
        NbNetCard=${#TabNetCard[*]}

        echo " Le nombre de carte est : $NbNetCard"
        echo -e "${vert} LES CARTES SONT : ${TabNetCard[*]}${blanc}"
        nmcli -f DEVICE device
        echo -e "Veuillez saisir le nom de votre carte reseau:\t\c"
        read NetCard
        trouve=0
        i=1
        #On va chercher si la carte saisie est dans le tableau des cartes
        while [ $i -lt $NbNetCard -a $trouve -eq 0 ]
        do
            lacarte=${TabNetCard[$i]}
            if [ "$NetCard" = "$lacarte" ]
            then
                trouve=1
            fi
            ((i++))
        done
        if [ $trouve -eq 1 ]
        then

```



```

        echo -e "Votre carte est : ${vert}$NetCard${blanc}"

        break

    fi

done

;;

esac

}

modicart()

{
    echo -e " Les paramètres renouvelés sont:"

    echo -e " Adresse IP \t $IP"

    echo -e " carte MASK \t $MASK"

    echo -e " carte GTW \t $GTW"

    echo -e " carte DNS \t $DNS"

    echo -e " La carte réseau va être purgée des anciens paramètres\t $NetCard"

    ip addr flush dev $NetCard

    ip addr add "$IP"/"$MASK" broadcast "$IPa.$IPb.$IPc.255" dev $NetCard

    ip route add default via "$GTW"

    ip a

    cible="/etc/resolv.conf"

    echo "#Modification du fichier resolv.conf" > $cible

    echo "nameserver $DNS"                >>$cible

}

ajoulogi()

{
    echo -e "${vert} Vérification de la connexion à internet${jaune}"

    sites="iprec.fr yahoo.fr"

    erreurs=0

```

```

for site in $sites
do
    ping -c4 $site >/dev/null 2>&1
    ((erreurs+=?$))
done

if [ $erreurs -eq 0 ]
then
    echo -e "${vert} Le reseau est ok${jaune}"
else
    echo -e "${rouge} Probleme de reseau ${blanc}"
    exit 13
fi

echo -e "${vert} Mise à jour de l'OS${jaune}"
yum -y update
echo -e "${vert} Ajout des packets nécessaires${blanc}"
packages="dhcp-server.x86_64 tftp-server.x86_64 syslinux.x86_64 nfs4-acl-tools.x86_64 nfs-utils"
erreurs=0
for packet in $packages
do
    yum -y install $packet
    ((erreurs+=?$))
done

# ameliorer si erreur !=0
echo -e "${vert} Vérification des packets${blanc}"
erreurs=0
for packet in $packages
do
    rpm -q $packet

```

```

    ((erreurs+=${?}))

done

echo -e "${vert} Vérification SELinux${blanc}"

sestatus
}

echo -e "${jaune}\n***** DEBUT *****${blanc}"

echo -e "${rouge}VMware peut présenter des bugs, dans ce cas redémarrer toute la machine${blanc}"

echo -e "${jaune}ATTENTION avec une distribution minimaliste, VSCode peut refuser de se connecter en SSH,"

echo -e " et des commandes comme VIM peuvent manquer par défaut.\n\n${blanc}"

# si EUID = 0 afficher en vert le script va s'executer et bascule jaune
# sinon en rouge, passer root pour executer le script, bascule jaune
# sortir du script

#verifier fonctions.sh est ordinaire et executable

#si oui, executer

#sinon en ROUGE, afficher fonctions.sh manquant ou non executable

#basculer en jaune

if [ -f fonctions.sh -a -x fonctions.sh ]
then
    . fonctions.sh
else
    echo -e "${rouge} Fichier fonctions.sh est manquant ou non executable ${blanc}"
    exit
fi

VerifRoot

ip -4 a

```

```
echo -e "${jaune}Entrez 11 configurer une IP en bridge"
echo -e " ou autre valeur pour sauter l'étape.${blanc}"
read reponse
if [ $reponse -eq 11 ]
then
    echo -e "Merci de configurer la carte réseau en bridge, puis ENTER"
    read reponse
    echo "***** ConfigNETCARD *****"
    ConfigNETCARD
    echo "***** ConfigIP *****"
    ConfigIP
    echo "***** ConfigMASK *****"
    ConfigMASK
    echo "***** ConfigGTW *****"
    ConfigGTW
    echo "***** ConfigDNS *****"
    ConfigDNS
    modicart
fi

echo -e "${jaune}Entrez 12 si vous voulez installer les compléments logiciels"
echo -e " ou autre valeur pour sauter l'étape.${blanc}"
read reponse
if [ $reponse -eq 12 ]
then
    ajoutlogi
fi

ip a
```



```

echo -e "${jaune}Entrez 13 pour configurer une ip pour serveur PXE"
echo -e " ou autre valeur pour sauter l'étape.${blanc}"
read reponse
if [ $reponse -eq 13 ]
then
    echo -e "Paramétrer avec les valeurs suivantes"
    echo -e "IP = 192.168.C.254"
    echo -e "MASQUE = 255.255.255.0"
    echo -e "PASSERELLE = 192.168.C.254"
    echo -e "DNS = 192.168.C.254"
    echo -e " où C est votre année de naissance (uniquement les deux derniers chiffres)"
    echo -e "Pour commencer:"
    echo -e "Merci de configurer la carte réseau en host-only, puis ENTER"
    read reponse
    echo "***** ConfigNETCARD ***** "
    ConfigNETCARD
    echo "***** ConfigIP ***** "
    ConfigIP
    echo "***** ConfigMASK ***** "
    ConfigMASK
    echo "***** ConfigGTW ***** "
    ConfigGTW
    echo "***** ConfigDNS ***** "
    ConfigDNS
    modicart
fi
echo -e "***** Rappel des paramètres actuels de la carte *****"
echo -e "${orange}IPa=$IPa IPb=$IPb IPc=$IPc IPd=$IPd${jaune}"
DNS=$DNSa.$DNSb.$DNSc.$DNSd
echo -e "${orange}DNSa=$DNSa DNSb=$DNSb DNSc=$DNSc DNSd=$DNSd${blanc}"

```

```
echo "$IPa.$IPb.$IPc.$IPd-$MASK-$GTW-$DNS"
```

```
echo -e "${jaune}Entrez 14 si vous voulez activer les services dhcp et tftp"
```

```
echo -e " ou autre valeur pour sauter l'étape (déconseillé).${blanc}"
```

```
read reponse
```

```
if [ $reponse -eq 14 ]
```

```
then
```

```
    cible2="/etc/dhcp/dhcpd.conf"
```

```
    echo "authoritative;" > $cible2
```

```
    echo "subnet $IPa.$IPb.$IPc.0 netmask $MASK {" >>$cible2
```

```
    echo "range $IPa.$IPb.$IPc.100 $IPa.$IPb.$IPc.110;" >>$cible2
```

```
    echo "option routers $GTW;" >>$cible2
```

```
    echo "option domain-name-servers $DNS;" >>$cible2
```

```
    echo "option broadcast-address $IPa.$IPb.$IPc.255;" >>$cible2
```

```
    echo "next-server $IP;" >>$cible2
```

```
    echo "option subnet-mask $MASK;" >>$cible2
```

```
    echo "filename \"pxelinux.0\";" >>$cible2
```

```
    echo "}" >>$cible2
```

```
echo "***** dhcpd.conf actuel"
```

```
cat /etc/dhcp/dhcpd.conf
```

```
systemctl restart dhcpd.service && systemctl enable dhcpd.service
```

```
systemctl status dhcpd.service
```

```
systemctl restart tftp.socket && systemctl enable tftp.socket
```

```
systemctl status tftp.socket
```

```
echo "***** copie de fichiers et création de dossier"
```

```
cp /usr/share/syslinux/{pxelinux.0,ldlinux.c32} /var/lib/tftpboot/
```

```

cp /usr/share/syslinux/ldlinux.c32 /var/lib/tftpboot/

[ -d /var/lib/tftpboot/pxelinux.cfg ] || mkdir /var/lib/tftpboot/pxelinux.cfg

[ -d /SrcAlma ] || mkdir /SrcAlma

[ -d /reponses ] || mkdir /reponses

fi

echo -e "${jaune}Entrez 15 si vous voulez connecter DVD avec fichier ISO"

echo -e " ou autre valeur pour sauter l'étape.${blanc}"

read reponse

if [ $reponse -eq 15 ]

then

# echo -e "${rouge} Vérifier que le lecteur est connecté avec un fichier ISO dedans${jaune}"

# echo -e "Appuyer sur entrée"

# read reponse

mount /dev/sr0 /SrcAlma/

cible="/var/lib/tftpboot/pxelinux.cfg/default"

echo "PROMPT 0" >$cible

echo "DEFAULT linux" >>$cible

echo "LABEL linux" >>$cible

echo "KERNEL vmlinuz" >>$cible

echo "APPEND initrd=initrd.img inst.ks=nfs:$IP:/reponses/ks.cfg" >>$cible

cp /SrcAlma/images/pxeboot/{vmlinuz,initrd.img} /var/lib/tftpboot/

cible="/etc/exports"

echo "/SrcAlma $IPa.$IPb.$IPc.0/24(ro,sync,no_root_squash,no_subtree_check)" >$cible

echo "/reponses $IPa.$IPb.$IPc.0/24(ro,sync,no_root_squash,no_subtree_check)" >>$cible

fi

echo -e "${jaune}Entrez 16 pour activer NFS"

echo -e " ou autre valeur pour sauter l'étape.${blanc}"

read reponse

```

```
if [ $reponse -eq 16 ]
then
    systemctl restart nfs-server.service && systemctl enable nfs-server.service
    systemctl status nfs-server.service

    exportfs -v
    firewall-cmd --permanent --add-service=tftp
    firewall-cmd --permanent --add-service=nfs
    firewall-cmd --permanent --add-service=rpc-bind
    firewall-cmd --permanent --add-service=mountd
    systemctl restart firewalld.service
fi
```

```
echo -e "${jaune}Entrez 17 pour remplacer ks.cfg pour distribution minimale"
echo -e "${jaune}Entrez 18 pour remplacer ks.cfg pour distribution maximale"
echo -e " ou autre valeur pour sauter l'étape.${blanc}"
read reponse
# Fichier Kickstart à modifier
```

```
if [ $reponse -eq 17 ]
then
    cp -f /root/anaconda-ks.cfg ks.cfg
    cp -f mini.cfg /reponses/ks.cfg
else
    if [ $reponse -eq 18 ]
    then
        cp -f /root/anaconda-ks.cfg ks.cfg
        cp -f maxi.cfg /reponses/ks.cfg
    fi
fi
```

```

echo -e "${orange}***** Script terminé *****"

echo -e "${blanc}***** L'Os peut être installé sur une nouvelle machine *****${blanc}"


#-----

#ks_file="ks.cfg"

# Vérifie si la ligne contient "cdrom" et remplace par la nouvelle ligne

# if grep -q "cdrom" "$ks_file"; then

#     sed -i 's|cdrom|nfs --server=192.168.72.254 --dir=/SrcAlma|' "$ks_file"

#     echo "La ligne 'cdrom' a été remplacée par 'nfs --server=192.168.72.254 --dir=/SrcAlma'."

# else

#     echo "La ligne 'cdrom' n'a pas été trouvée dans le fichier."

#fi

```

Script fonctions

```

VerifRoot()

{

    if [ $EUID -eq 0 ]

    then

        echo "***** Root ok, le script va s'executer *****"

    else

        echo "*****passer root pour executer le script *****"

        exit

    fi

# echo "test suite"

}

```


VerifOctet()

```
{
    debut=$1
    fin=$2
    octet=$3
    while true
    do
        if [ $octet -ge $debut -a $octet -le $fin ]
        then
            echo -e "La valeur ${vert} $octet ${blanc} est ok"
            return $octet
            break
        else
            echo -e "La valeur ${rouge} $octet ${blanc} est fausse"
            echo -e "Entrez une valeur comprise entre $debut et $fin:\t\c"
            read octet
        fi
    done
}
```

SaisiIp()

```
{
    echo -e "Entrez votre adresse ip sous la forme a.b.c.d :\t\c"
    read ip

    echo "Votre adresse est : $ip"
    IFS="."
    while true
    do
        TabIp=($ip)
```

```

NbOctets=${#TabIp[*]}

echo "Le nombre d octet est : $NbOctets"

if [ $NbOctets -ne 4 ]
then
    echo -e "${rouge}$ip ${blanc} Veuillez saisir une adresse ip valide (4 octets) :\t\c"
    read ip
else
    octeta=${TabIp[0]}; octetb=${TabIp[1]}; octetc=${TabIp[2]}; octetd=${TabIp[3]}
    IFS=" "
    break
fi

echo "Votre adresse est : $ip"
done
}

```

ConfigIP()

```

{
    #Recherche de l'adresse ip active

    ip=$(ip -4 addr | grep $NetCard | grep inet | awk -F "/" '{print $6}')

    if [ -n "$ip" ]
    then
        echo -e "L'adresse active est :[${vert}$ip${blanc}]"
        echo -e "Confirmez vous cette adresse ip O/n ? : \t\c"
        read reponse
    else
        reponse="non"
    fi

    echo "VOus avez tape : ---${reponse}+++"
}

```

```

case "$reponse" in
"" | [oO] ) IFS="."
    TabIp=($ip)
    IPa=${TabIp[0]}; IPb=${TabIp[1]}; IPc=${TabIp[2]}; IPd=${TabIp[3]}
    IP=$IPa.$IPb.$IPc.$IPd
    echo -e "${vert}IPa=$IPa IPb=$IPb IPc=$IPc IPd=$IPd${blanc}"
    IFS=" ";;
* ) SaisiIp

```

```

# 1 <= octeta <= 254
VerifOctet 1 254 $octeta
octeta=$?
IP=$octeta
echo -e "${jaune} IP=$IP ${blanc}"

```

```

# 0 <= octetb <= 255
VerifOctet 0 255 $octetb
octetb=$?
IP+=.$octetb
echo -e "${jaune} IP=$IP ${blanc}"

```

```

# 0 <= octetc <= 255
VerifOctet 0 255 $octetc
octetc=$?
IP+=.$octetc
echo -e "${jaune} IP=$IP ${blanc}"

```

```

# 1 <= octetd <= 254
VerifOctet 1 254 $octetd

```

```

        octetd=$?

        IP+=.$octetd

        echo -e "${jaune} IP=$IP ${blanc}"

        IPa=$octeta; IPb=$octetb; IPc=$octetc; IPd=$octetd

        echo -e "${vert}IPa=$IPa IPb=$IPb IPc=$IPc IPd=$IPd${blanc}"

        IFS=" "

    esac

}

NbBit2Decimal ()
{
    NbBit=$1 #NbBit=4

    valeur=128

    octet=0

    while [ $NbBit -gt 0 ] #4>0?          3>0          2>0          1>0          0>0
    do

        ((octet=octet + valeur)) #octet=0+128=128  octet=128+ 64=192          octet=192+32=224
        octet=224+16=240

        ((valeur=valeur / 2)) #valeur=128/2=64  valeur=64/2=32          valeur=32/2=16
        valeur=16/2=8

        ((NbBit=NbBit - 1)) #NbBit=4-1=3  NbBit=3-1=2  NbBit=2-1=1          NbBit=1-1=0

    done

    echo "conversion en $octet"

    return $octet
}

SaisieMASK()
{
    cidr=$1

    while true

```

```

do
    case $cidr in
        [0-8] )
            NbBit2Decimal $cidr

            MASKa=$?; MASKb=0; MASKc=0; MASKd=0;
            MASK=$MASKa.$MASKb.$MASKc.$MASKd

            break;;

        9|1[0-6] )
            ((cidr=cidr-8))

            NbBit2Decimal $cidr

            MASKb=$?; MASKa=255; MASKc=0; MASKd=0;
            MASK=$MASKa.$MASKb.$MASKc.$MASKd

            break;;

        1[7-9]|2[0-4] )
            ((cidr=cidr-16))

            NbBit2Decimal $cidr

            MASKc=$?; MASKa=255; MASKb=255; MASKd=0;
            MASK=$MASKa.$MASKb.$MASKc.$MASKd

            break;;

        2[5-9]|3[0-2] )
            ((cidr=cidr-24))

            NbBit2Decimal $cidr

            MASKd=$?; MASKa=255; MASKb=255; MASKc=255;
            MASK=$MASKa.$MASKb.$MASKc.$MASKd

            break;;

        * )
            echo -en "Votre saisie ${rouge} ($cidr) est fausse ${blanc}"

            echo -e " saisir votre masque en notation CIDR VALEUR COMPRIS ENTRE 0 ET 32 : \tc"

            read cidr;;

    esac

done

}

```

ConfigMASK()

```
{
    cidr=$(ip -4 addr | grep $NetCard | grep inet | awk -F ' / ' '{print $7}')
    if [ -n "$cidr" ]
    then
        echo -e "Mask en notation CIDR [${vert}$cidr${blanc}]"
        echo -e "Confirmez vous le mask O/n ?:${jaune}\\t\\c"
        read reponse
        echo -e "${blanc}"
    else
        reponse="non"
    fi
    case "$reponse" in
        "" | [oO] ) SaisieMASK $cidr ;;
        * ) echo -e "Masque en notation CIDR svp ${jaune}\\t\\c"
            read cidr
            echo -e "${blanc}"
            SaisieMASK $cidr ;;
    esac
}
```

ConfigGTW()

```
{
    #Recherche de l'adresse GTW active
    ip=$(ip -4 route show | grep "^default" | awk 'NR == 1 {print $3}')
    # ip=$(ip -4 addr | grep $NetCard | grep inet | awk -F " / " '{print $6}')
    if [ -n "$ip" ]
    then
        echo -e "L'adresse passerelle est :[${vert}$ip${blanc}]"
    fi
}
```



```

echo -e "Confirmez vous cette adresse ip O/n ?: \t\c"

read reponse

else

    reponse="non"

fi

echo "Vous avez choisi : ---${reponse}+++"

case "$reponse" in

    "" | [oO] ) IFS="."

        TabIp=($ip)

        GTWa=${TabIp[0]}; GTWb=${TabIp[1]}; GTWc=${TabIp[2]}; GTWd=${TabIp[3]}

        GTW=$GTWa.$GTWb.$GTWc.$GTWd

        echo -e "${vert}GTWa=$GTWa GTWb=$GTWb GTWc=$GTWc GTWd=$GTWd${blanc}"

        IFS=" ";

    * ) SaisiIp

        # 1 <= octeta <= 254

        VerifOctet 1 254 $octeta

        octeta=$?

        GTW=$octeta

        echo -e "${jaune} GTW=$GTW ${blanc}"

        # 0 <= octetb <= 255

        VerifOctet 0 255 $octetb

        octetb=$?

        GTW+=.$octetb

        echo -e "${jaune} GTW=$GTW ${blanc}"

        # 0 <= octetc <= 255

        VerifOctet 0 255 $octetc

        octetc=$?

```

```

        GTW+=.$octetc
        echo -e "${jaune} GTW=$GTW ${blanc}"

        # 1 <= octetd <= 254
        VerifOctet 1 254 $octetd
        octetd=$?
        GTW+=.$octetd
        echo -e "${jaune} GTW=$GTW ${blanc}"
        GTWa=$octeta; GTWb=$octetb; GTWc=$octetc; GTWd=$octetd
        echo -e "${vert}GTWa=$GTWa GTWb=$GTWb GTWc=$GTWc GTWd=$GTWd${blanc}"
        IFS=" "

    esac
}

ConfigDNS()
{
    #Recherche de l'adresse dns active
    # ip=$(ip -4 addr | grep $NetCard | grep inet | awk -F "/" '{print $6}')
    ip=$(grep "^nameserver" /etc/resolv.conf | awk 'NR == 1 {print $2}')
    if [ -n "$ip" ]
    then
        echo -e "L'adresse active est :[${vert}$ip${blanc}]"
        echo -e "Confirmez vous cette adresse ip O/n ?: \t\c"
        read reponse
    else
        reponse="non"
    fi
    echo "Vous avez saisi : ---${reponse}+++"
    case "$reponse" in
        "" | [oO] ) IFS="."

```

```

TabIp=($ip)
DNSa=${TabIp[0]}; DNSb=${TabIp[1]}; DNSc=${TabIp[2]}; DNSd=${TabIp[3]}
DNS=$DNSa.$DNSb.$DNSc.$DNSd
echo -e "${vert}DNSa=$DNSa DNSb=$DNSb DNSc=$DNSc DNSd=$DNSd${blanc}"
IFS=" ";
* ) SaisiIp

```

```

# 1 <= octeta <= 254
VerifOctet 1 254 $octeta
octeta=$?
DNS=$octeta
echo -e "${jaune} DNS=$DNS ${blanc}"

```

```

# 0 <= octetb <= 255
VerifOctet 0 255 $octetb
octetb=$?
DNS+=.$octetb
echo -e "${jaune} DNS=$DNS ${blanc}"

```

```

# 0 <= octetc <= 255
VerifOctet 0 255 $octetc
octetc=$?
DNS+=.$octetc
echo -e "${jaune} DNS=$DNS ${blanc}"

```

```

# 1 <= octetd <= 254
VerifOctet 1 254 $octetd
octetd=$?
DNS+=.$octetd
echo -e "${jaune} DNS=$DNS ${blanc}"

```

```

        DNSa=$octeta; DNSb=$octetb; DNSc=$octetc; DNSd=$octetd
        echo -e "${vert}DNSa=$DNSa DNSb=$DNSb DNSc=$DNSc DNSd=$DNSd${blanc}"
        IFS=" "
    esac
}

```

Fichier de configuration mini.cfg

```

# mini.cfg
# Generated by Anaconda 34.25.5.9
# Generated by pykickstart v3.32
#version=RHEL9
# Use graphical install
graphical

%addon com_redhat_kdump --enable --reserve-mb='auto'

%end

# Keyboard layouts
keyboard --xlayouts='fr (oss)'
# System language
lang fr_FR.UTF-8

# Use CDROM installation media
#cdrom

#use nfs installation
nfs --server=192.168.72.254 --dir=/SrcAlma

```

%packages

@^minimal-environment

%end

Run the Setup Agent on first boot

firstboot --enable

Generated using Blivet version 3.6.0

ignoredisk --only-use=nvme0n1

Partition clearing information

clearpart --none --initlabel

clearpart --all --initlabel

Disk partitioning information

part swap --fstype="swap" --ondisk=nvme0n1 --size=4000

part / --fstype="xfs" --ondisk=nvme0n1 --size=10000

part /home --fstype="xfs" --ondisk=nvme0n1 --size=1000

System timezone

timezone Europe/Paris --utc

Root password

rootpw --iscrypted

\$6\$5n0lkmHWZ8NhPVlv\$3CDCIAeLq7U03sKrkdE.nnOeYzndFMII6G.e3J8xIOawJWWeOokEtAGcf9NtNR
QXSEnqfpQ1IEIqOkOOEILGY/

user --name=cedrik --

password=\$6\$L8ltuMTk2Efk10sV\$W4e9Bd8LRgosCDGU5zFPykLinrmBSQGUKFS8uRyEDIS8sy0qxGjonu
8OxftC09NPBvHRY4KXbLXU07KB2C2Kk/ --iscrypted --gecos="cedrik"

[Fichier de configuration maxi.cfg](#)

maxi.cfg

```
# Generated by Anaconda 34.25.5.9

# Generated by pykickstart v3.32

#version=RHEL9

# Use graphical install

graphical

# repo --name="AppStream" --baseurl=file:///run/install/sources/mount-0000-cdrom/AppStream

%addon com_redhat_kdump --enable --reserve-mb='auto'

%end


# Keyboard layouts

keyboard --xlayouts='fr (oss)'

# System language

lang fr_FR.UTF-8


#use nfs installation

nfs --server=192.168.72.254 --dir=/SrcAlma


%packages

@^graphical-server-environment

%end


# Run the Setup Agent on first boot

firstboot --enable


# Generated using Blivet version 3.6.0

ignoredisk --only-use=nvme0n1
```

Partition clearing information

clearpart --all --initlabel

Disk partitioning information

part swap --fstype="swap" --ondisk=nvme0n1 --size=1024

part / --fstype="xfs" --ondisk=nvme0n1 --grow --size=1

part /home --fstype="xfs" --ondisk=nvme0n1 --size=2048

System timezone

timezone Europe/Paris --utc

Root password

rootpw --iscrypted

\$6\$I28uUFIHFPCSZw2I\$I0wCpIDDXM07WAPLdUUs0en1PX1I7E/GSCtHjdjWFDwhYwu9rkI3aC4e7iOyKS
7SdB3XOIDQoiUdcG7r/S6sH0

user --name=cedric --

password=\$6\$6qMggKqx5uD6CT6/\$47pm6uMA4eZAJi0d6Jadn1rvmfpy7Lc7C4BZoRjenZwclyIS5gtMHtFS
LW5y.8.GY.gIXiPayxUvmAmwG1zMP/ --iscrypted --gecos="cedric"